

Introdução à ciência e engenharia dos sistemas complexos

**Tópico 0x.0y – Laboratório: Módulo linguístico semântico local do SimServ
A “IA” local**

Carlos Ignacio Z. Mammanna

Como especificar as tarefas do Módulo linguístico-semântico local

Módulo linguístico-semântico local

Serviço local de processamento de linguagem

Camada local de processamento linguístico por LLM

No servidor do laboratório do curso, SimServ, o Módulo linguístico-semântico local opercionalizado pelo conjunto de software Ollama, é usado para:

- analisar narrativas;**
- extrair entidades, agentes, variáveis e relações;**
- gerar JSON estruturado;**
- sintetizar pseudocódigo;**
- reescrever especificações;**
- comparar descrições;**
- transformar texto em estrutura operacional;**
- auxiliar na geração de código.**

Nos notebooks, o módulo linguístico-semântico local é usado para transformar uma narrativa em uma especificação estruturada do experimento computacional.

O SimServ utiliza um serviço local de processamento linguístico por LLM, executado via Ollama, para realizar análise semântica, síntese textual, geração de especificações JSON e apoio à geração de código.

O Ollama é o serviço local que executa modelos LLM; as tarefas de análise, síntese e estruturação linguística são definidas pelo notebook por meio de prompts, funções de validação e formatos de saída.

O SimServ utiliza o Ollama como servidor local de modelos LLM, integrado a um conjunto de ferramentas de análise e síntese linguístico-semântica.

Para nomes de funções ou mensagens no notebook:

```
analisar_narrativa()  
gerar_especificacao_json()  
sintetizar_pseudocodigo()  
avaliar_consistencia_semantica()  
reescrever_especificacao()  
chamar_servico_llm_local()
```

O que o Ollama executa

chamadas locais de inferência usando modelos LLM

O Ollama é o servidor local de inferência de modelos LLM. No SimServ, ele é usado como base para uma camada de processamento linguístico-semântico, responsável por análise de narrativas, extração estruturada, síntese textual, geração de especificações e apoio à geração de código.

Ele é o serviço que recebe uma requisição, carrega ou usa um modelo local e devolve uma resposta gerada pelo modelo.

Para poder funcionar o Ollama precisa de informações específicas. Estas especificações são expressas no código elaborado para utilizar as funções do Ollama. Esse código monta o processo de como as funções do Ollama são chamadas e as condições de suas execução.

Conceitos importantes:

API

Endpoint

API

API significa Application Programming Interface, ou Interface de Programação de Aplicações.

É um conjunto de regras que define como um programa deve pedir algo a outro programa e como receberá a resposta.

Exemplo:

curl http://ollama:11434/api/tags

um notebook, script ou terminal está pedindo ao Ollama uma informação.

Endpoint

Um endpoint é um endereço específico dentro de uma API.

É como uma “porta de atendimento” para uma tarefa específica.

Exemplos do Ollama:

/api/tags Lista os modelos instalados

/api/generate Gera uma resposta a partir de um prompt

/api/chat Faz uma conversa estruturada

/api/ps Mostra modelos carregados na memória

/api/show Mostra detalhes de um modelo

http://ollama:11434/api/generate

significa:

http://ollama:11434 → endereço do serviço Ollama;

/api/generate → endpoint específico para gerar uma resposta.

Função técnica	Endpoint/API	Uso no SimServ
Geração textual direta	<code>`/api/generate`</code>	Transformar um prompt em resposta textual, JSON, pseudocódigo ou código
Conversa estruturada	<code>`/api/chat`</code>	Manter mensagens com papéis: sistema, usuário, assistente
Saída JSON/estruturada	<code>`format: "json"`</code> ou JSON Schema	Gerar especificações formais para notebooks e simulações
Embeddings	<code>`/api/embed`</code>	Busca semântica, RAG, comparação de textos, agrupamento de narrativas
Listar modelos locais	<code>`/api/tags`</code>	Saber quais modelos estão instalados
Ver modelos carregados	<code>`/api/ps`</code>	Saber quais modelos estão residentes na memória
Inspecionar modelo	<code>`/api/show`</code>	Ver detalhes, parâmetros, template, família, quantização e capacidades
Ver versão do serviço	<code>`/api/version`</code>	Diagnóstico de ambiente

Tarefas típicas no SimServ

Tarefa do SimServ	O que o modelo faz
analisar_narrativa()	Identifica fenômeno, agentes, ambiente, variáveis, mecanismos e lacunas
gerar_especificacao_json()	Converte a narrativa em estrutura formal validável
sintetizar_pseudocodigo()	Produz passos computacionais intermediários
gerar_codigo_base()	Gera uma primeira versão de código Python/NetLogo
avaliar_consistencia()	Verifica se a especificação é coerente com a narrativa
reescrever_especificacao()	Torna o texto mais claro ou mais operacional
gerar_embedding_textual()	Transforma textos em vetores para busca/comparação semântica

Codificação típica a usar nos notebooks JupyterLab

```
executar_tarefa_llm_local()  
chamar_servico_llm_local()  
analisar_narrativa_com_llm()  
gerar_especificacao_semantica()  
gerar_saida_estruturada()  
registrar_metricas_inferencia()
```

O que o Ollama informa sobre seu funcionamento

O Ollama já devolve várias métricas úteis. Elas não explicam “como o modelo pensou”, mas informam como a chamada foi executada.

A documentação do Ollama lista essas métricas de uso e informa que os tempos são medidos em nanossegundos. A documentação de `/api/generate` também mostra os campos `done_reason`, `total_duration`, `load_duration`, `prompt_eval_count`, `prompt_eval_duration`, `eval_count`, `eval_duration` e `logprobs`.

Indicador	Cálculo	Interpretação
Tokens de saída por segundo	$\text{eval_count} / \text{eval_duration} * 1\text{e}9$	Velocidade de geração
Tokens de entrada por segundo	$\text{prompt_eval_count} / \text{prompt_eval_duration} * 1\text{e}9$	Velocidade de leitura/processamento do prompt
Tempo total em segundos	$\text{total_duration} / 1\text{e}9$	Latência total da tarefa
Tempo de carregamento em segundos	$\text{load_duration} / 1\text{e}9$	Custo de carregar modelo na memória
Fração do tempo carregando modelo	$\text{load_duration} / \text{total_duration}$	Indica se o atraso vem do carregamento
Fração do tempo gerando resposta	$\text{eval_duration} / \text{total_duration}$	Indica se o atraso vem da geração
Tamanho da entrada	<code>prompt_eval_count</code>	Complexidade/tokenização do prompt
Tamanho da resposta	<code>eval_count</code>	Quantidade de texto produzido

Indicadores de qualidade: sinais indicadores indiretos de qualidade operacional

Ollama informa métricas de execução.

Ollama não informa, automaticamente, se a resposta é “verdadeira”, “boa” ou “pedagogicamente adequada”.

Sinal	O que indica
JSON válido	A resposta respeitou o formato esperado
JSON compatível com schema	A resposta tem os campos obrigatórios
done_reason == "stop"	A geração terminou normalmente
done_reason == "length" ou equivalente	A resposta pode ter sido cortada
eval_count muito pequeno	Pode indicar resposta vazia ou insuficiente
eval_count muito grande	Pode indicar verbosidade ou perda de foco
prompt_eval_count muito alto	Prompt talvez esteja longo demais
logprobs, se usados	Podem indicar confiança token a token, mas não verdade factual
repetição entre chamadas	Pode indicar estabilidade ou instabilidade da tarefa
validação externa por regras	Confere se campos, unidades, variáveis e mecanismos fazem sentido

Formato JSON

Ollama também aceita format: "json" ou um JSON Schema para forçar saídas estruturadas, o que é especialmente útil para transformar narrativas em especificações formais. Isso é muito importante no SimServ, porque permite avaliar a resposta não apenas pelo texto, mas por critérios verificáveis:

```
campos_obrigatorios = [  
  "fenomeno",  
  "objetivo",  
  "agentes",  
  "ambiente",  
  "variaveis",  
  "parametros",  
  "mecanismos",  
  "lacunas",  
  "pseudocodigo"  
]
```

não, em vez de perguntar “a IA respondeu bem?”, o notebook pode perguntar:

A saída foi JSON válido?

Todos os campos obrigatórios existem?

Os parâmetros aparecem com unidades?

Os agentes têm atributos?

Os mecanismos estão descritos como regras operacionais?

Há lacunas explicitadas?

O pseudocódigo é executável ou ainda narrativo?

Isso transforma uma “resposta oracular” em uma resposta auditável.

O que o Ollama informa sobre modelos e recursos

Essas informações são obtidas através do estado do serviço

Modelos disponíveis

/api/tags lista os modelos locais disponíveis e retorna nome, tamanho, data de modificação, digest e detalhes como família, tamanho de parâmetros e quantização.

**quais modelos estão instalados;
quais são modelos de código;
quais são modelos gerais;
quais são leves ou pesados;
qual quantização está sendo usada.**

Modelos carregados

/api/ps lista os modelos atualmente carregados em memória e informa, entre outros campos, size, expires_at e size_vram.

**qual modelo está residente;
se um modelo ainda está ocupando memória;
quanto de VRAM aparece associado ao modelo;
quando ele será descarregado pelo keep_alive.**

Informações fornecidas pelo Ollama

Informações do modelo

/api/show mostra informações como detalhes do modelo, modelfile, template, parâmetros, licença e prompt de sistema. Em respostas detalhadas, também podem aparecer capacidades do modelo, como completion e vision

**qual família de modelo está sendo usada;
qual quantização;
qual tamanho;
se tem capacidade de visão;
se é modelo adequado para código, texto geral ou embeddings;
quais parâmetros estão definidos no Modelfile.**

Vocabulário para o SimServ

Situação	Termo recomendado
Chamada simples ao Ollama	chamada de inferência local
Uso técnico do modelo	execução local de modelo LLM
Uso didático em narrativa	processamento linguístico-semântico local
Geração de JSON	extração estruturada por LLM local
Comparação de textos	análise semântica vetorial
Geração de código	síntese assistida de código por LLM local
Diagnóstico do serviço	telemetria da inferência local
Medição de desempenho	métricas de execução da inferência

Estrutura recomendada para notebooks do SimServ

Produto 1 — Resultado da tarefa

Produto 2 — Registro técnico da inferência

Inferência significa, de modo geral, tirar uma conclusão a partir de informações disponíveis.

**inferência estatística;
intervalo de confiança;
teste de hipótese;
estimação de parâmetros.**

**entrada: prompt do usuário
modelo: LLM já treinado
saída: texto, JSON, código, resumo, classificação etc.**

“Ollama é um servidor local de inferência”

O Ollama recebe uma entrada textual, envia essa entrada para um modelo LLM local já instalado e devolve uma saída gerada pelo modelo.

Registro técnico da inferência

```
{  
  "modelo": "qwen2.5-coder:14b",  
  "tarefa": "analisar_narrativa",  
  "tempo_total_s": 12.4,  
  "tempo_carregamento_s": 2.1,  
  "tokens_prompt": 820,  
  "tokens_resposta": 460,  
  "tokens_por_segundo": 38.2,  
  "done_reason": "stop",  
  "json_valido": true,  
  "schema_valido": true,  
  "campos_ausentes": []  
}
```

Esse segundo bloco é essencial para o SimServ, porque permite acompanhar:

**se o modelo está lento;
se o prompt ficou grande demais;
se a resposta foi cortada;
se o modelo carregou na GPU;
se o JSON foi válido;
se o modelo é adequado para a tarefa;
se modelos diferentes têm desempenho diferente;
se a atividade pode ser usada em aula com vários alunos simultâneos.**

Exemplo: Resultado da tarefa

Arquivo JSON contendo:

```
{  
  "fenomeno": "...",  
  "objetivo": "...",  
  "agentes": [...],  
  "ambiente": "...",  
  "variaveis": [...],  
  "parametros": [...],  
  "mecanismos": [...],  
  "lacunas": [...],  
  "pseudocodigo": [...]  
}
```